

Plano de Testes

Modernizacao de Sistema Legado - Cooperativa Genesis

ProjetoProjeto Final COBOL - Acelera Maker / Montreal

Componente	API de Cadastro de Clientes (.NET + COBOL + MySQL)
Autor	Davi
Data	Julho de 2026

Este documento descreve o plano de testes da solucao de modernizacao do cadastro de clientes da Cooperativa Genesis, contemplando os cenarios funcionais de consulta (GET) e atualizacao (PUT), os criterios de aceitacao de cada cenario, os resultados esperados e obtidos, alem das evidencias de execucao via Swagger UI e dos testes automatizados xUnit.

1. Escopo dos Testes

O plano cobre a validação funcional dos dois endpoints expostos pela API ClientesApi, responsáveis por consultar e atualizar dados cadastrais de clientes, com processamento delegado ao componente legado COBOL (CLIENTES.exe) e persistência no banco MySQL (BankSystem).

Endpoint	Metodo	Descricao
/api/Clientes/{codigo}	GET	Consulta os dados cadastrais de um cliente pelo código
/api/Clientes/{codigo}	PUT	Atualiza telefone e e-mail de um cliente existente

2. Ambiente de Testes

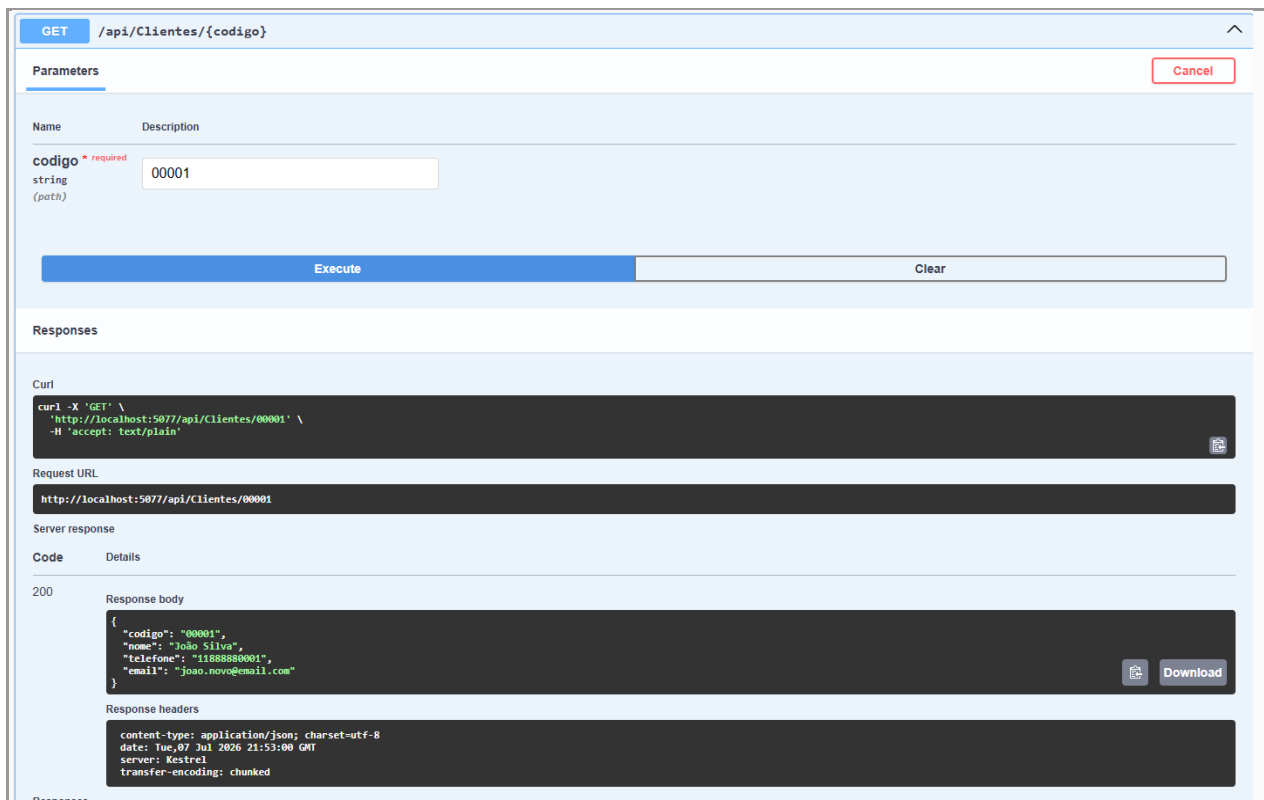
Item	Detalhe
API	ASP.NET Core 8 - http://localhost:5077
Ferramenta de teste manual	Swagger UI
Testes automatizados	xUnit + Moq
Banco de dados	MySQL - banco BankSystem, tabela clientes
Componente legado	GnuCOBOL 2.0 (32 bits) - CLIENTES.exe

3. Casos de Teste Funcionais

CT-01 - Consulta de cliente existente (GET)

Endpoint	GET /api/Clientes/{codigo}
Pre-condicao	Cliente com código válido cadastrado na tabela clientes do MySQL.
Passos	1. Enviar requisição GET informando um código de cliente existente. 2. Observar o status HTTP e o corpo da resposta.
Critério de aceitacao	A API deve retornar HTTP 200 com JSON contendo código, nome, telefone e e-mail do cliente.
Resultado esperado	HTTP 200 - JSON com os dados cadastrais completos do cliente.
Resultado obtido	HTTP 200 - JSON retornado com código, nome, telefone e e-mail corretos. OK.

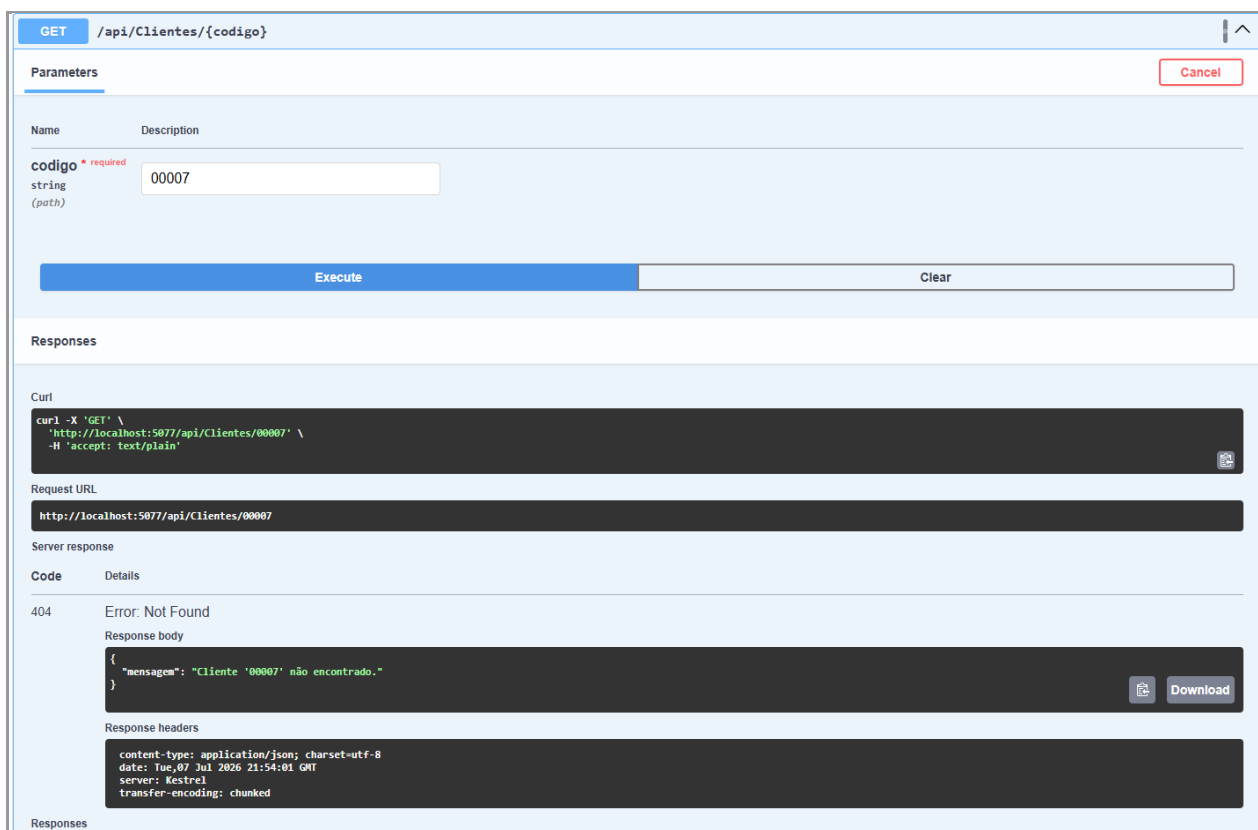
Evidencia: Print do Swagger UI mostrando a requisição GET e a resposta 200 com o JSON do cliente.



CT-02 - Consulta de cliente inexistente (GET)

Endpoint	GET /api/Clientes/{codigo}
Pre-condicao	Codigo informado nao existe na tabela clientes.
Passos	1. Enviar requisicao GET informando um codigo que nao existe no banco. 2. Observar o status HTTP e o corpo da resposta.
Critério de aceitacao	A API deve informar de forma clara que o cliente nao foi encontrado, sem expor erro interno.
Resultado esperado	HTTP 404 - mensagem indicando cliente nao encontrado (correspondente ao retorno NOTFOUND do COBOL).
Resultado obtido	HTTP 404 - mensagem de cliente nao encontrado retornada corretamente. OK.

Evidencia: Print do Swagger UI mostrando a requisicao GET com codigo inexistente e a resposta 404.



CT-03 - Atualizacao de telefone e e-mail (PUT) - sucesso

Endpoint	PUT /api/Clientes/{codigo}
Pre-condicao	Cliente com codigo valido cadastrado na tabela clientes.
Passos	1. Enviar requisicao PUT informando codigo existente e novo telefone/e-mail. 2. Verificar o status HTTP retornado. 3. Consultar novamente o cliente (GET) para confirmar a persistencia da alteracao.
Criterio de aceitacao	A API deve atualizar os dados no banco e retornar confirmacao de sucesso.
Resultado esperado	HTTP 200 - confirmacao de atualizacao (correspondente ao retorno ATUALIZADO do COBOL); consulta subsequente deve refletir os novos valores.
Resultado obtido	HTTP 200 - atualizacao confirmada. Consulta subsequente (GET) retornou telefone e e-mail atualizados. OK.

Evidencia: Print do Swagger UI mostrando a requisicao PUT com sucesso e o GET subsequente confirmando a alteracao.

PUT

/api/Clientes/{codigo}

Parameters

Cancel

Reset

Name	Description
codigo * required string (path)	00001

Request body

application/json

```
{
  "telefone": "31988887777",
  "email": "joao.silva.novo@email.com"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5077/api/Clientes/00001' \
  -H 'accept: text/plain' \
  -H 'content-type: application/json' \
  -d '{
    "telefone": "31988887777",
    "email": "joao.silva.novo@email.com"
  }'
```

Request URL

http://localhost:5077/api/Clientes/00001

Server response

Code	Details
200	Response body <pre>{ "codigo": "00001", "nome": "João Silva", "telefone": "31988887777", "email": "joao.silva.novo@email.com" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 07 Jul 2026 21:56:09 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

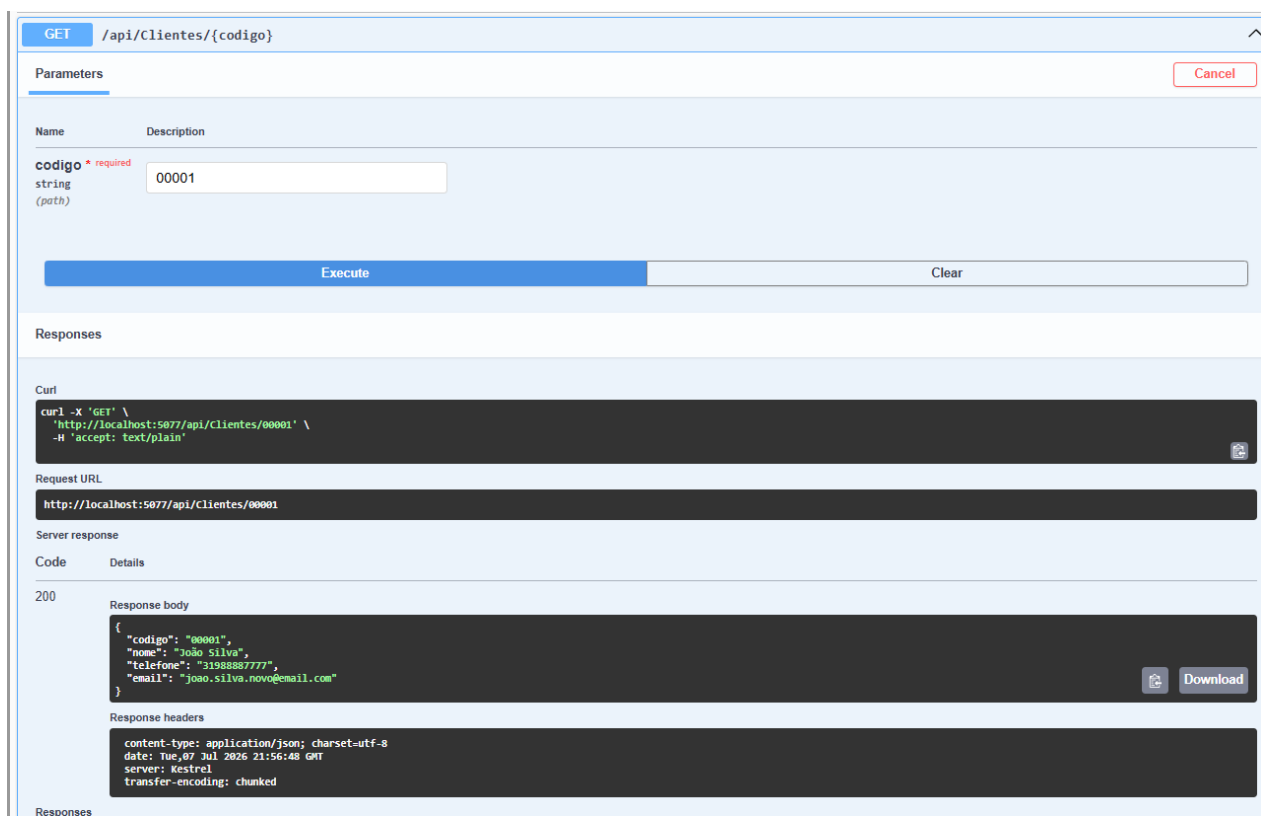
Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{
  "codigo": "string",
  "nome": "string",
  "telefone": "string",
  "email": "string"
}
```



CT-04 - Atualizacao de cliente inexistente (PUT)

Endpoint	PUT /api/Cientes/{codigo}
Pre-condicao	Codigo informado nao existe na tabela clientes.
Passos	1. Enviar requisicao PUT informando um codigo que nao existe no banco. 2. Observar o status HTTP e o corpo da resposta.
Critério de aceitacao	A API nao deve realizar nenhuma alteracao e deve informar que o cliente nao foi encontrado.
Resultado esperado	HTTP 404 - mensagem indicando cliente nao encontrado, nenhuma alteracao realizada no banco.
Resultado obtido	HTTP 404 - mensagem de cliente nao encontrado retornada corretamente. Nenhum registro foi alterado. OK.

Evidencia: Print do Swagger UI mostrando a requisicao PUT com codigo inexistente e a resposta 404.

Clientes

GET /api/Clientes/{codigo}

PUT /api/Clientes/{codigo}

Parameters

Cancel

Reset

Name	Description
codigo * required string (path)	99999

Request body

application/json

```
{  "telefone": "3198887777",  "email": "joao.silva.novo@email.com"}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  'http://localhost:5077/api/Clientes/99999' \  -H 'accept: text/plain' \  -H 'Content-Type: application/json' \  -d '{  "telefone": "3198887777",  "email": "joao.silva.novo@email.com"  }'
```

Request URL

```
http://localhost:5077/api/Clientes/99999
```

Server response

Code	Details
404	Error: Not Found

Response body

```
{  "mensagem": "Cliente '99999' não encontrado."}
```

Download

Response headers

```
content-type: application/json; charset=utf-8  date: Tue, 07 Jul 2026 21:57:30 GMT  server: Kestrel  transfer-encoding: chunked
```

Responses

Code	Description	Links
------	-------------	-------

CT-05 - Falha de comunicacao com o componente COBOL

Endpoint	PUT /api/Clientes/{codigo} (ou GET)
Pre-condicao	Simulacao de erro no processo COBOL (ex.: retorno inesperado via STDOUT ou falha na execucao do CLIENTES.exe).
Passos	1. Forcar um cenario de erro na chamada ao processo COBOL (via mock no teste automatizado). 2. Verificar o tratamento de erro pela API.

Criterio de aceitacao	A API deve tratar a falha de forma controlada, retornando um erro apropriado (HTTP 500) sem expor detalhes internos sensiveis, correspondente ao retorno ERRO do COBOL.
Resultado esperado	HTTP 500 - mensagem de erro generica e controlada, sem quebra da aplicacao.
Resultado obtido	Validado via teste automatizado xUnit com mock simulando falha do COBOL. Comportamento de erro tratado corretamente. OK (teste automatizado - ver secao 4).

Evidencia: Este cenario e validado por teste automatizado (mock), nao requer print do Swagger.

```
PS C:\FinalProject\legacy-client-modernization> cd testes\ClientesApi.Testes && dotnet test --filter "FullyQualifiedName=ClientesApi.Tests.ClientesControllerTests.Put_ErroCOBOL_Retorna500" --verbosity normal 2>&
1
Restauração concluída (0,5s)
ClientesApi net8.0 êxito (0,4s) → C:\FinalProject\legacy-client-modernization\api\ClientesApi\bin\Debug\net8.0\ClientesApi.dll
ClientesApi.Tests net8.0 êxito (3,4s) → bin\Debug\net8.0\ClientesApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.246990445aia (64-bit .NET 8.0.25)
[xUnit.net 00:00:00.53] Discovering: ClientesApi.Tests
[xUnit.net 00:00:00.56] Discovered: ClientesApi.Tests
[xUnit.net 00:00:00.57] Starting: ClientesApi.Tests
fail: ClientesApi.Controllers.ClientesController[0]
Erro COBOL na atualiza#o: ERRO|DADOS_INVALIDOS
[xUnit.net 00:00:01.51] Finished: ClientesApi.Tests
ClientesApi.Tests teste net8.0 êxito (2,7s)

Resumo do teste: total: 1; falhou: 0; bem-sucedido: 1; ignorado: 0; duração: 2,7s
Construir êxito em 7,5s
PS C:\FinalProject\legacy-client-modernization\testes\ClientesApi.Testes>
```

4. Testes Automatizados (xUnit)

Alem dos testes funcionais manuais via Swagger, foi implementado um conjunto de testes automatizados com xUnit e Moq no projeto ClientesApi. Testes, cobrindo tanto a logica unitaria quanto cenarios de integracao com mocks do componente COBOL.

Categoria	Quantidade	Cobertura
Testes unitarios	3	Regras de negocio e mapeamento de dados na camada de servico
Testes de integracao (mockada)	5	Endpoints GET e PUT, incluindo cenarios de sucesso, nao encontrado e falha do COBOL
Total	8	Todos os 8 testes executam com sucesso (dotnet test)

Observacao tecnica: o log de execucao do dotnet test pode exibir uma entrada de fail isolada relativa ao teste que simula erro do componente COBOL; trata-se de comportamento esperado da simulacao, e todos os 8 testes sao reportados como aprovados ao final da execucao.

Evidencia: print do resultado da execucao do comando dotnet test.

```
PS C:\FinalProject\legacy-client-modernization\testes\ClientesApi.Testes> dotnet test
Restauração concluída (0,5s)
ClientesApi net8.0 êxito (0,3s) -> C:\FinalProject\legacy-client-modernization\api\ClientesApi\bin\Debug\net8.0\ClientesApi.dll
ClientesApi.Testes net8.0 êxito (0,2s) -> bin\Debug\net8.0\ClientesApi.Testes.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 8.0.25)
[xUnit.net 00:00:00.08] Discovering: ClientesApi.Testes
[xUnit.net 00:00:00.11] Discovered: ClientesApi.Testes
[xUnit.net 00:00:00.11] Starting: ClientesApi.Testes
fail: ClientesApi.Controllers.ClientesController[0]
      Erro COBOL na atualiza#o: ERRO|DADOS_INVALIDOS
[xUnit.net 00:00:00.58] Finished: ClientesApi.Testes
ClientesApi.Testes teste net8.0 êxito (1,3s)

Resumo do teste: total: 8; falhou: 0; bem-sucedido: 8; ignorado: 0; duração: 1,3s
Construir êxito em 2,9s
PS C:\FinalProject\legacy-client-modernization\testes\ClientesApi.Testes>
```

5. Resumo dos Resultados

Caso	Descricao	Resultado
CT-01	Consulta de cliente existente	OK
CT-02	Consulta de cliente inexistente	OK
CT-03	Atualizacao com sucesso	OK
CT-04	Atualizacao de cliente inexistente	OK
CT-05	Falha de comunicacao com o COBOL	OK (via mock)

xUnit	Suite automatizada (8 testes)	8/8 aprovados
-------	-------------------------------	---------------

Conclusao: todos os cenarios funcionais previstos foram executados com sucesso, tanto manualmente via Swagger quanto por meio da suite automatizada xUnit. A solucao atende aos requisitos funcionais de consulta, atualizacao e tratamento de cenarios de execucao (cliente nao encontrado e falha do componente legado).